ASSIGNMENT

Introduction to Hash Cryptography
Cryptography and Network Security

Q1. Define hash cryptography. Explain the fundamental principles and techniques of cryptographic hash functions with suitable examples.

Hash cryptography uses a mathematical function to convert input data of any practical length into a fixed-length value called a hash, digest, or message digest. A cryptographic hash function is designed to be easy to compute but extremely difficult to reverse or find two different inputs with the same digest. Unlike encryption, hashing does not use a key in its basic form and is not intended to recover the original message.

Important principles include: (1) **Deterministic output:** the same input always produces the same digest. (2) **Fixed length:** inputs of different sizes produce digests of a specified size. (3) **Preimage resistance:** given a digest, it should be impractical to find an input that produces it. (4) **Second-preimage resistance:** given one input, it should be impractical to find a different input with the same digest. (5) **Collision resistance:** it should be impractical to find any two different inputs with the same digest. (6) **Avalanche effect:** a small change in input should cause a large, unpredictable change in output.

A hash function processes data in blocks using operations such as bitwise logical functions, modular addition, rotations, permutations and compression rounds. Many designs use an iterative construction: the message is padded, divided into blocks, and each block updates an internal state. SHA-256, for example, processes message blocks and produces a 256-bit digest. A file can be hashed and its digest compared with a trusted digest to detect changes.

```
Input message → Padding and block division → Hash rounds / compression → Fixed-length
                                                                        message digest
```

Q2. Discuss the role of cryptographic hash functions in cryptographic applications.

Data integrity verification: A sender or publisher provides a trusted hash of a file. The receiver calculates the hash of the received file and compares both values. If they differ, the file may have changed. The hash itself must come from a trusted source; an attacker who can replace both the file and its published hash can defeat a simple comparison.

Digital signatures: Instead of signing a large document directly, a signing system normally hashes the document and signs the digest using a private signing key. The receiver verifies the signature with the public key and independently hashes the received document. This makes signing efficient and helps detect modification.

Password storage: Websites should not store plain-text passwords. They store password hashes produced by a password-hashing scheme such as Argon2id, scrypt, bcrypt or PBKDF2, with a unique salt for each password. These schemes are deliberately slower or memory-intensive to make large-scale guessing harder. A fast general-purpose hash such as plain SHA-256 should not be used alone for password storage.

Message authentication: A keyed hash-based message authentication code (HMAC) combines a secret key with a hash function. A receiver with the same secret key can verify that a message has not been changed and that it came from someone who knows the key. HMAC-SHA-256 is a common example. A plain hash alone does not prove who created a message.

Q3. Explain advantages and disadvantages of hash cryptography and the properties of a secure hash function.

Advantages: Hashing is efficient; it produces a compact fixed-size digest; it helps detect accidental or intentional data changes; it supports digital signatures and authentication constructions; and it is useful for indexing, checksums and secure data structures.

Disadvantages: Hashing is one-way and cannot recover lost original data. A digest alone does not authenticate its source. Collisions are theoretically possible because many possible inputs map to a finite output space. Weak or outdated algorithms may be vulnerable to practical attacks. Passwords hashed with a fast hash can be guessed quickly if attackers obtain the hash database.

Preimage resistance: Given a target digest h, it should be computationally infeasible to find message m such that H(m)=h. **Second-preimage resistance:** Given a particular message m1, it should be infeasible to find another

message m2 where H(m1)=H(m2). **Collision resistance:** It should be infeasible to find any distinct pair m1 and m2 with equal hashes. Collision resistance is especially important for digital signatures and certificates.

The avalanche effect means that changing even one input bit should change many output bits in an apparently unpredictable way. A secure modern hash should also avoid structural weaknesses and be resistant to known cryptanalytic attacks.

Q4. Describe the SHA family and compare SHA-1, SHA-2 and SHA-3.

The Secure Hash Algorithm (SHA) family is a group of standardized cryptographic hash functions. It was developed for integrity protection and is used in digital signatures, certificates and security protocols.

SHA-1: Produces a 160-bit digest. It was widely used in older systems, but practical collision attacks have been demonstrated. It is no longer considered secure for digital signatures or other collision-sensitive uses and should not be selected for new systems.

SHA-2: Includes SHA-224, SHA-256, SHA-384 and SHA-512, among other variants. The numbers indicate digest length in bits. SHA-256 and SHA-512 are widely used and remain accepted for many security applications. SHA-2 uses a Merkle–Damgård-style construction with different internal parameters for its variants.

SHA-3: Based on the Keccak design and standardized as a different construction from SHA-2. It uses a sponge construction, which absorbs input into an internal state and then squeezes out the digest. SHA3-224, SHA3-256, SHA3-384 and SHA3-512 provide different output lengths. SHA-3 offers design diversity alongside SHA-2.

```
SHA-1: 160-bit (deprecated for secure signatures) | SHA-2: 224/256/384/512-bit | SHA-3: 224/256/384/512-bit sponge family
```

Q5. Explain applications of SHA algorithms in cryptographic protocols and information security.

Digital certificates: Certificate systems use hash functions to calculate fingerprints and to support certificate-signature processes. A certificate authority signs certificate data, and a verifier checks the signature and certificate chain. Current systems use secure hash algorithms such as SHA-256 rather than SHA-1 for collision-sensitive signing.

Digital signatures: A SHA digest represents the document being signed. The signature algorithm signs the digest or a structured representation of it. If the document changes, its digest changes and signature verification fails.

File integrity checking: Software publishers may publish SHA-256 checksums. Users calculate the downloaded file's hash and compare it with the value obtained from an authentic publisher channel.

Secure communication: SHA-2 and SHA-3 variants are used in protocol components, HMAC constructions, key derivation functions and certificate systems. For example, TLS uses hash functions in transcript hashing and authentication-related operations. HMAC can authenticate records or messages when a shared secret is used.

Note: For password storage, use a dedicated password-hashing function such as Argon2id, scrypt, bcrypt or PBKDF2 rather than plain SHA algorithms.